Ministerul Educaţiei și Cercetării al Republicii
MoldovaUniversitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory work 2:
Study and empirical analysis of sorting algorithms. Analysis of QuickSort, MergeSort, HeapSort, ShellSort

Elaborated:
st. gr. FAF-241 Pleșu Dinu

Verified:
asist. univ. Fiștic Cristofor

Chişinău - 2026

TABLE OF CONTENTS

ALGORITHM ANALYSIS

Objective

Study and analyze different sorting algorithms

Tasks:

1 Implement the algorithms listed above in a programming language
2 Establish the properties of the input data against which the analysis is performed
3 Choose metrics for comparing algorithms
4 Perform empirical analysis of the proposed algorithms
5 Make a graphical presentation of the data obtained
6 Make a conclusion on the work done.

Theoretical Notes

An alternative to mathematical analysis of complexity is empirical analysis.

This may be useful for: obtaining preliminary information on the complexity class of an algorithm; comparing the efficiency of two (or more) algorithms for solving the same problems; comparing the efficiency of several implementations of the same algorithm; obtaining information on the efficiency of implementing an algorithm on a particular computer.

In the empirical analysis of an algorithm, the following steps are usually followed:

1.The purpose of the analysis is established.

2.Choose the efficiency metric to be used (number of executions of an operation (s) or time execution of all or part of the algorithm.

3.The properties of the input data in relation to which the analysis is performed are established (data size or specific properties).

4.The algorithm is implemented in a programming language.

5.Generating multiple sets of input data.

6.Run the program for each input data set.

7. The obtained data are analyzed.

The choice of the efficiency measure depends on the purpose of the analysis. If, for example, the aim is to obtain information on the complexity class or even checking the accuracy of theoretical estimate then it is appropriate to use the number of operations performed. But if the goal is to assess the behavior of the implementation of an algorithm then execution time is appropriate.

After the execution of the program with the test data, the results are recorded and, for the purpose of the analysis, either synthetic quantities (mean, standard deviation, etc.) are calculated or a graph with appropriate pairs of points (i.e. problem size, efficiency measure) is plotted.

IMPLEMENTATION

Quick Sort:

The recursive method of Quicksort follows a divide-and-conquer strategy, where the problem is reduced at each step by partitioning the input array into smaller subarrays. Unlike the Fibonacci algorithm, which recomputes the same values multiple times, QuickSort avoids redundant work by splitting the dataset and processing independent parts. However, it still relies on recursion, meaning the function repeatedly calls itself until base conditions are met.

Algorithm Description:

QuickSort operates by selecting a pivot element and rearranging the array such that all elements smaller than the pivot are placed to its left, and all elements greater than the pivot are placed to its right. This partitioning step is followed by recursively applying the same process to the left and right subarrays. The recursion stops when subarrays contain zero or one element, as they are already sorted.

```
quicksort(A, low, high):
        if low < high:
            pivotIndex = partition(A, low, high)
            quicksort(A, low, pivotIndex - 1)
            quicksort(A, pivotIndex + 1, high)
        partition(A, low, high):
        pivot = A[high]
        i = low - 1
        for j from low to high - 1:
            if A[j] <= pivot:a
               i = i + 1
               swap A[i], A[j]
       swap A[i + 1], A[high]
       return i + 1
```

Implementation:

```python
def quick_sort(array):
    if len(array) <= 1:
        return array
    pivot_index = random.randint(0, len(array) - 1)  # Pick a random index
    pivot = array[pivot_index]

    # Partition the array into three parts: less than, equal to, and greater than the pivot
    left = [x for x in array if x < pivot]
    middle = [x for x in array if x == pivot]
    right = [x for x in array if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)
```

Figure 1:Quick Sort in Python

Results:

After running the QuickSort function for each input size defined in the dataset specifications and measuring execution time and memory usage, the following results were obtained. The experiments were conducted across multiple dataset types, including random, nearly sorted, reverse sorted, and datasets with limited unique values.

	Dataset Type	Array Size	Execution Time (seconds)	Peak Memory (MB)
0	Random Large Dataset	100000	1.253026	4.532776
1	Nearly Sorted Dataset	100000	1.115404	3.312401
2	Small Dataset	100000	0.000000	0.002480
3	Integer Limited Range (1-100)	100000	0.099067	4.267166
4	Floating-Point Dataset	100000	1.205260	4.082863

Figure 2: Quicksort Execution time and Peak Memory usage depending on context

In Figure 2 is represented the table of results for the tested input sizes. The first row denotes the array sizes for which the algorithm was executed. The following rows present the execution times measured in seconds from the start to the completion of the sorting process and peak memory usage. It can be observed that QuickSort maintains relatively low execution times compared to other algorithms, especially for large random datasets.

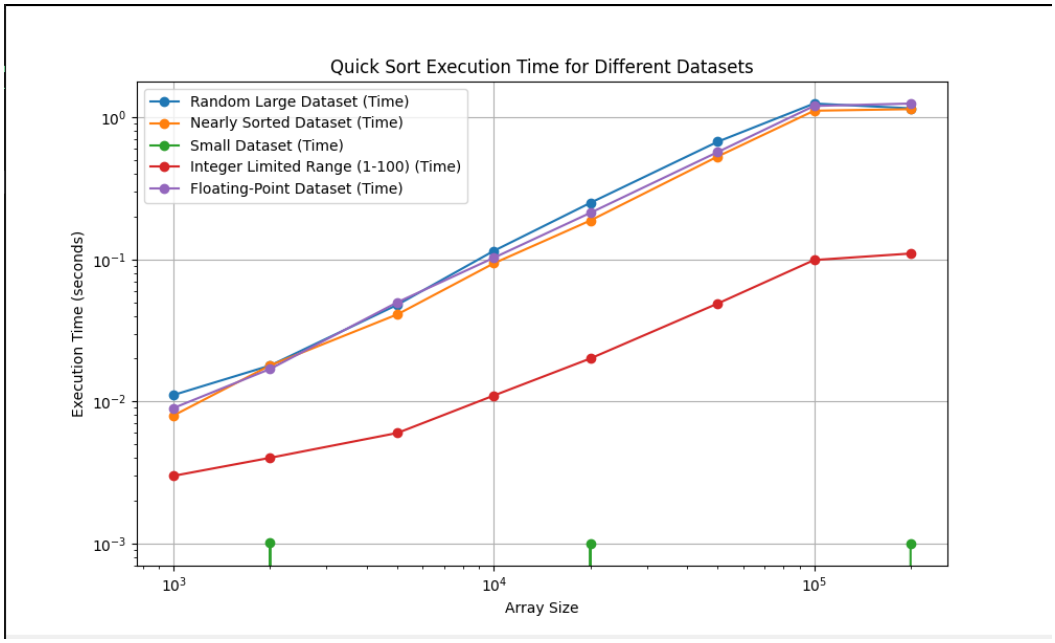

Figure 3: Quick Sort Execution Time Graph

Furthermore, in Figure 3, which illustrates the growth of execution time relative to input size and type, the curve demonstrates a near linearithmic progression. This indicates that the algorithm follows an average-case time complexity of O(n log n).

The results confirm that QuickSort is highly efficient in practice, particularly for large datasets and inputs with many duplicate values. The use of random pivot selection significantly reduces the likelihood of encountering the worst-case scenario. However, when poor pivot choices occur, such as consistently selecting the smallest or largest element, the recursion depth increases and the time complexity degrades to $O(n^2)$.

Conclusion:

QuickSort achieves high performance due to its divide-and-conquer nature and efficient partitioning process. Its recursive structure does not lead to redundant computations, making it significantly more efficient than naïve recursive algorithms like Fibonacci. The empirical results align with theoretical expectations, confirming an average-case complexity of O(n log n) and demonstrating strong scalability across various input types.

Merge Sort:

The recursive method of MergeSort follows a divide-and-conquer approach, similar in structure to QuickSort, but with a key difference in how subproblems are handled and combined. Instead of partitioning around a pivot, MergeSort divides the array into equal halves and then merges the sorted results. This guarantees consistent performance, as the same operations are performed regardless of input order.

Algorithm Description:

MergeSort works by recursively splitting the input array into two halves until subarrays of size one are obtained. These subarrays are inherently sorted. The algorithm then combines these smaller sorted arrays through a merging process, producing larger sorted arrays until the entire dataset is reconstructed in sorted order.

The recursive MergeSort method follows the algorithm shown in the next pseudocode.

```
MergeSort(A):
    If length(A) ≤ 1:
       Return A
    Otherwise:
       middle = length(A) // 2
       left = MergeSort(A[0 : middle])
       right = MergeSort(A[middle : end])
       Return Merge(left, right)
    Merge(left, right):
    Initialize empty array result
    i = 0, j = 0
    While i < length(left) and j < length(right):
       If left[i] ≤ right[j]:
          Append left[i] to result
          i = i + 1
       Else:
```

Append right[j] to result
j = j + 1

Implementation:

```python
def merge_sort(array):
    if len(array) <= 1:
        return array

    middle = len(array) // 2
    left_array = array[:middle]
    right_array = array[middle:]

    return merge(merge_sort(left_array), merge_sort(right_array))

def merge(left, right):
    sorted_array = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            sorted_array.append(left[i])
            i += 1
        else:
            sorted_array.append(right[j])
            j += 1
    return sorted_array + left[i:] + right[j:]
```

Figure 4:MergeSort in Python

Results:

After running the MergeSort function for each input size defined in the dataset specifications and recording execution time and memory usage, the following results were obtained. The algorithm was tested on multiple dataset types, including random, nearly sorted, reverse sorted, and datasets with duplicate values.

	Dataset Type	Array Size	Execution Time (seconds)	Peak Memory (MB)
0	Random Large Dataset	100000	3.109117	3.815910
1	Nearly Sorted Dataset	100000	1.868931	3.475555
2	Small Dataset	100000	0.000000	0.001923
3	Integer Limited Range (1-100)	100000	3.107808	3.815674
4	Floating-Point Dataset	100000	3.271274	3.815674

Figure 5: MergeSort Execution time and Peak Memory usage depending on context

In Figure 5 is represented the table of results for the tested input sizes. The first row denotes the array sizes for which the algorithm was executed. The following rows display the execution times measured in seconds and peak memory usage. It can be observed that MergeSort maintains stable performance across all input types, with execution times growing consistently as the input size increases.

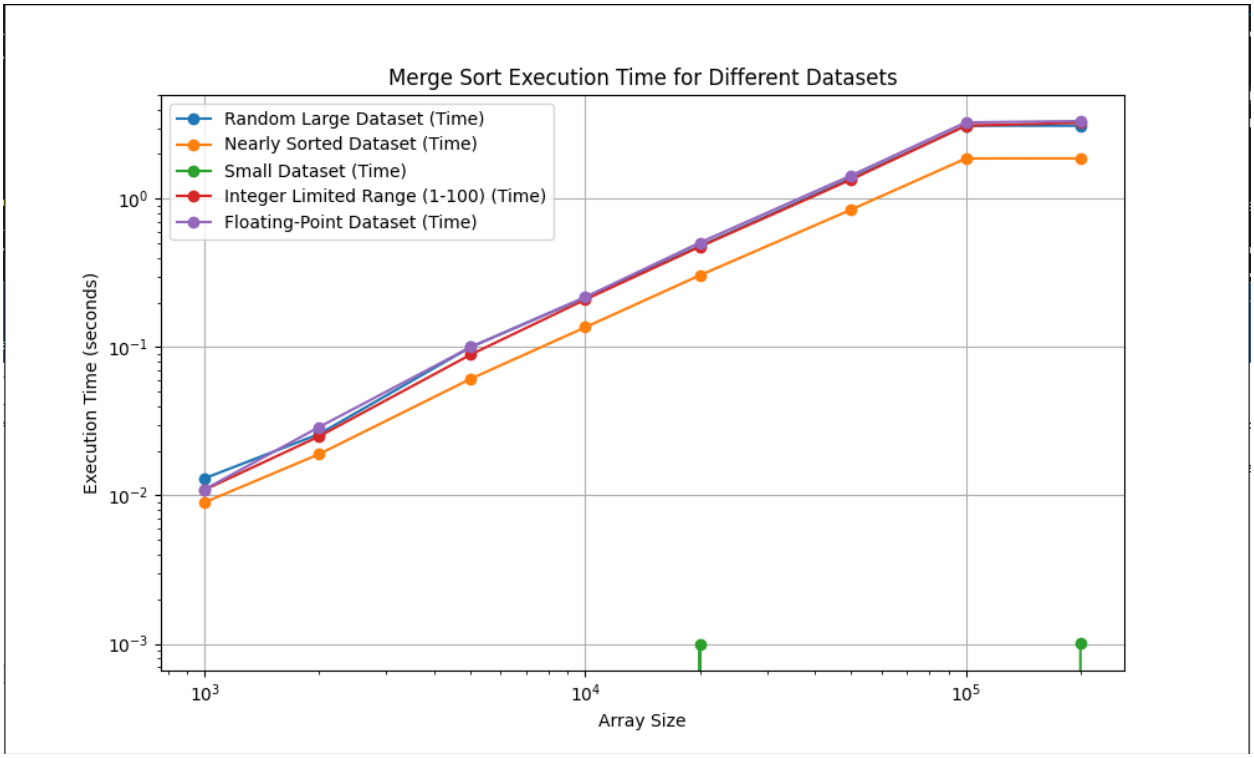

Figure 6: MergeSort Execution Time Graph

Furthermore, in Figure 6, which illustrates the relationship between execution time and input size for each input type, the curve shows a steady linearithmic growth. This reflects the theoretical time complexity of O(n log n), which remains constant regardless of the initial arrangement of the data. Unlike QuickSort, there are no spikes caused by unfavorable input configurations.

Conclusion:

MergeSort is an efficient and stable sorting algorithm with guaranteed performance across all input scenarios. Its recursive structure ensures that the problem size is reduced systematically without dependence on input characteristics. Although it uses more memory compared to in-place algorithms, its reliability and stability make it a preferred choice in applications where consistent execution time and order preservation are required.

HeapSort:

The method of Heapsort is not purely recursive in the same sense as QuickSort or MergeSort, but it can still be expressed using recursive heapify operations. It is based on the binary heap data structure, where the array is first transformed into a max heap, and then elements are extracted one by one to produce a sorted sequence. Unlike divide-and-conquer algorithms, HeapSort does not split the problem into independent subproblems, but instead repeatedly reorganizes the same structure.

Algorithm Description:

HeapSort works in two main phases. First, the input array is converted into a max heap, where each parent node is greater than its children. Second, the largest element, located at the root of the heap, is swapped with the last element of the array. The heap size is reduced, and the heap property is restored using the heapify process. This procedure is repeated until the entire array is sorted.

The recursive HeapSort method follows the algorithm shown in the next pseudocode.

The HeapSort method follows the algorithm shown in the next pseudocode.

```
HeapSort(A):
    n = length(A)
    For i from n // 2 - 1 down to 0:
    Heapify(A, n, i)
For i from n - 1 down to 1:
   Swap A[0] with A[i]
   Heapify(A, i, 0)
Heapify(A, n, i):
largest = i
left = 2 * i + 1
right = 2 * i + 2
If left < n and A[left] > A[largest]:
   largest = left
If right < n and A[right] > A[largest]:
   largest = right
If largest ≠ i:
   Swap A[i] with A[largest]
   Heapify(A, n, largest)
```

Implementation:

```python
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)

    # Build a max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements from the heap one by one
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]  # Swap
        heapify(arr, i, 0)
```

Figure 7:HeapSort in Python

Results:

After running the HeapSort function for each input size defined in the dataset specifications and recording execution time and memory usage, the following results were obtained. The algorithm was tested on various dataset types, including random, nearly sorted, reverse sorted, and datasets with duplicate values.

	Dataset Type	Array Size	Execution Time (seconds)	Peak Memory (MB)
0	Random Large Dataset	100000	4.305477	0.000751
1	Nearly Sorted Dataset	100000	4.490517	0.000729
2	Small Dataset	100000	0.000000	0.000076
3	Integer Limited Range (1-100)	100000	4.121961	0.000729
4	Floating-Point Dataset	100000	3.988184	0.000729

Figure 8: HeapSort Execution time and Peak Memory usage depending on context

In Figure 8 is represented the table of results for the tested input sizes. The first row denotes the array sizes for which the algorithm was executed. The following rows present the execution times measured in seconds and peak memory usage. It can be observed that HeapSort maintains consistent performance across all datasets, without significant variation caused by input order.

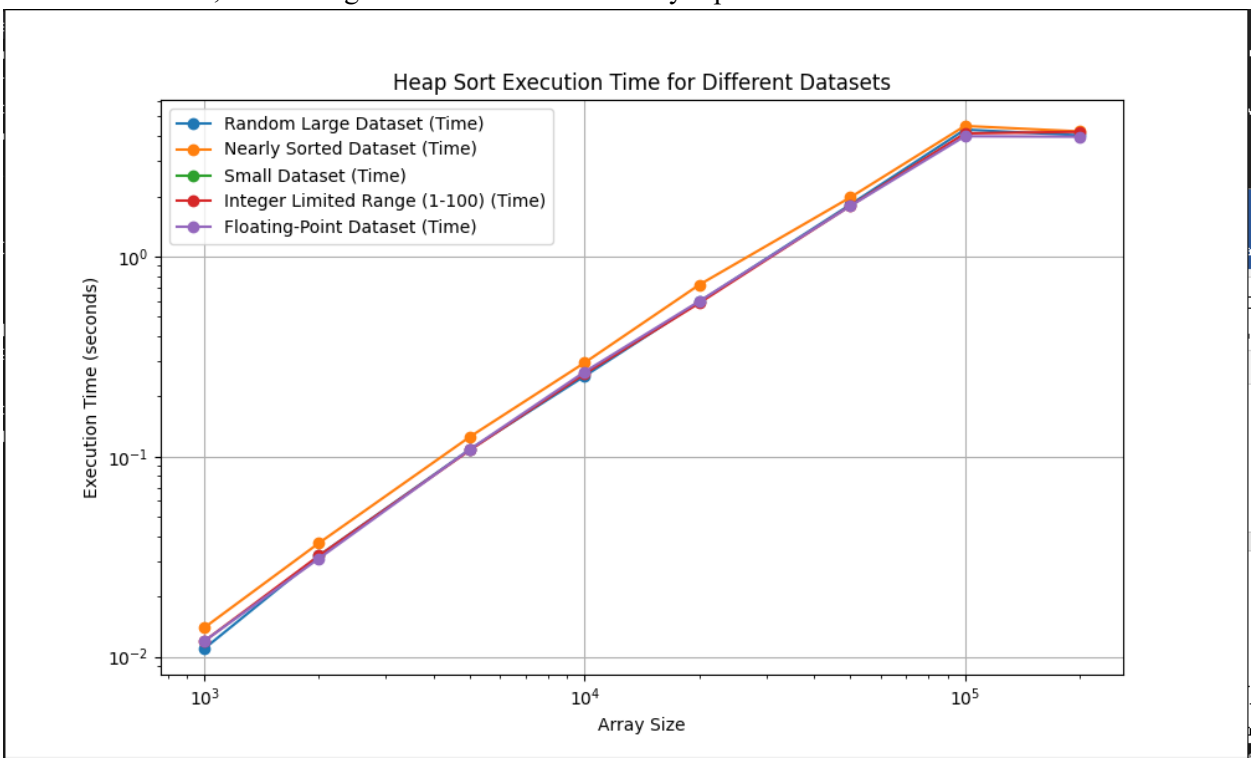

Figure 9: HeapSort Execution Time Graph

Furthermore, in Figure 9, which illustrates the growth of execution time relative to input size, the curve shows a steady increase following a linearithmic trend. This confirms the theoretical time complexity of O(n log n) for all cases. Unlike QuickSort, HeapSort does not suffer from worst-case degradation, and unlike MergeSort, it does not require additional memory for merging.

Conclusion:

HeapSort is an efficient sorting algorithm with guaranteed O(n log n) performance and minimal memory usage. Its structure avoids the risk of worst-case time complexity degradation and makes it suitable for memory-constrained environments. Although it is generally slower than QuickSort, its stability in performance and low space requirements make it a valuable alternative in scenarios where memory efficiency is critical.

ShellSort:

The method of Shellsort is based on an optimization of insertion sort, designed to improve performance by allowing exchanges of elements that are far apart. Unlike QuickSort and MergeSort, it does not rely on recursion, but instead uses a decreasing gap sequence to progressively reduce disorder in the array. This makes it more efficient than simple insertion sort, especially for medium-sized datasets.

Algorithm Description:

ShellSort works by dividing the array into subarrays defined by a gap value. Elements that are a certain gap apart are compared and sorted using a modified insertion sort. The gap is gradually reduced until it becomes 1, at which point the algorithm performs a final insertion sort pass. Because the array is already partially sorted by earlier passes, this final step is efficient.

The ShellSort method follows the algorithm shown in the next pseudocode.

```
ShellSort(A):
    n = length(A)
    gap = n // 2
    While gap > 0:
      For i from gap to n - 1:
        temp = A[i]
        j = i
        While j ≥ gap and A[j - gap] > temp:
          A[j] = A[j - gap]
          j = j - gap
        A[j] = temp
      gap = gap // 2
```

Implementation:

```python
def shell_sort(array):
    n = len(array)
    gap = n // 2
    while gap > 0:
        for i in range(gap, n):
            temp = array[i]
            j = i
            while j >= gap and array[j - gap] > temp:
                array[j] = array[j - gap]
                j -= gap
            array[j] = temp
        gap //= 2
    return array
```

Figure 10:ShellSort in Python

Results:

After running the ShellSort function for each input size defined in the dataset specifications and recording execution time and memory usage, the following results were obtained. The algorithm was tested on multiple dataset types, including random, nearly sorted, reverse sorted, and datasets with duplicate

values..

	Dataset Type	Array Size	Execution Time (seconds)	Peak Memory (MB)
0	Random Large Dataset	100000	10.030142	0.000248
1	Nearly Sorted Dataset	100000	2.427016	0.000195
2	Small Dataset	100000	0.000000	0.000076
3	Integer Limited Range (1-100)	100000	5.032050	0.000225
4	Floating-Point Dataset	100000	11.076402	0.000225

Figure 11: ShellSort Execution time and Peak Memory usage depending on context

In Figure 11 is represented the table of results for the tested input sizes. The first row denotes the array sizes for which the algorithm was executed. The following rows present the execution times measured in seconds and memory usage. It can be observed that ShellSort performs relatively well on smaller datasets, but execution time increases more rapidly compared to other algorithms as the input size grows.

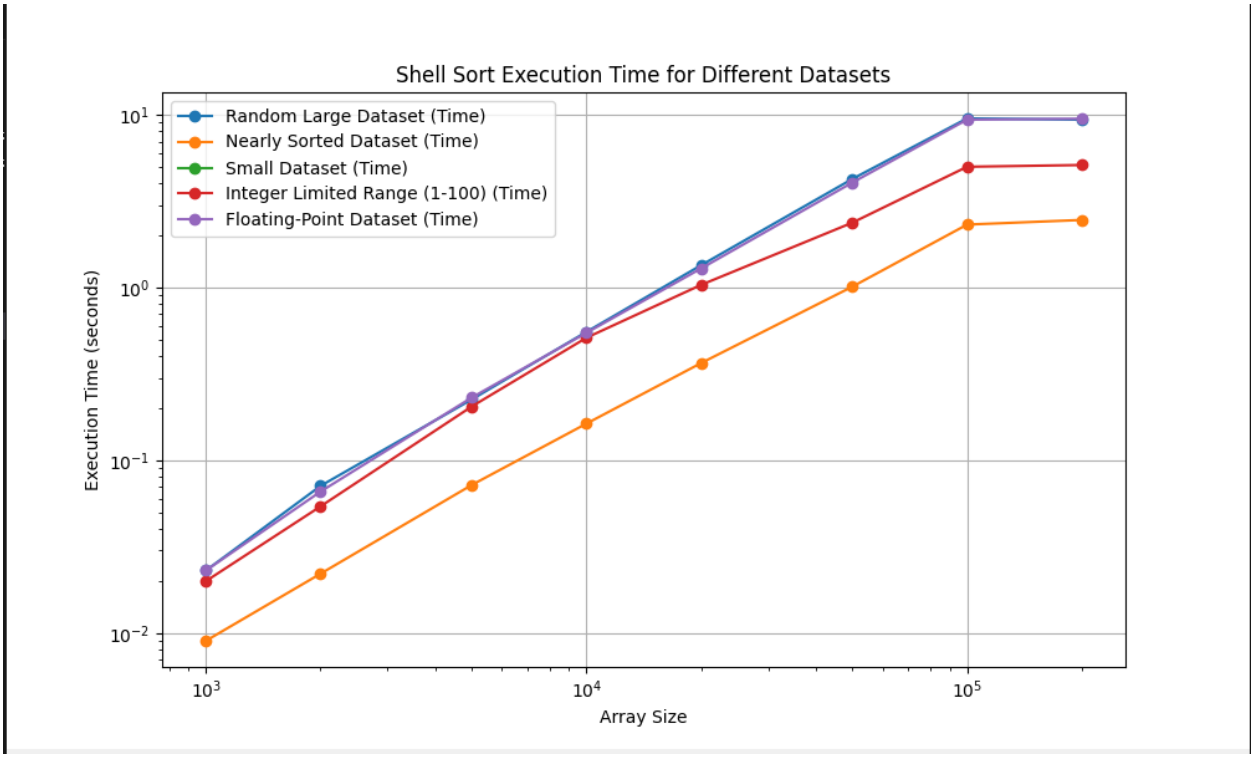

Figure 12: Shell Execution Time Graph

In Figure 12, which illustrates the growth of execution time relative to input size, the curve shows a steeper increase compared to O(n log n) algorithms. This reflects the approximate time complexity of O(n^1.25), which depends on the chosen gap sequence. Unlike MergeSort and HeapSort, performance is not strictly bounded, and unlike QuickSort, efficiency varies significantly with input characteristics.

Conclusion:

ShellSort is a simple and memory-efficient sorting algorithm that performs well on small to medium-sized datasets. Its in-place nature and low overhead make it suitable for scenarios where simplicity and memory usage are more important than optimal time complexity. However, due to its less predictable performance and higher time complexity for large inputs, it is generally not preferred for large-scale sorting tasks compared to more advanced algorithms.

CONCLUSION

This study analyzed Quicksort, MergeSort, Heapsort, and Shellsort through theoretical and empirical evaluation. Results show that performance depends not only on complexity but also on input data and implementation.

QuickSort achieved the best overall performance, with fast average execution and good scalability. MergeSort provided stable and predictable O(n log n) behavior in all cases. HeapSort was the most memory-efficient due to in-place execution, but slower in practice. ShellSort performed well on small or nearly sorted datasets but scaled poorly for large inputs.

The experiments confirmed that input characteristics strongly affect performance, and that simpler algorithms can outperform complex ones on small datasets. In conclusion, algorithm choice depends on context, QuickSort for speed, MergeSort for stability, HeapSort for low memory, and ShellSort for small or partially sorted data.

The understanding of the Algorithms is useful, to decide what should be implemented to better fit the needs, whether faster speeds or lower memory usage, or a combination of both, as systems don’t have unlimited resources.

References

1. Github Repo https://github.com/Prince-of-Nothing/aa-course-repo/tree/main/Lab2